◆Assignment 06

Assignment 6 — Tasks, Async & Resource Management

Modules 10–11 · 10 points

 Deadline: Week 6 — Thursday, 11:59 PM

Scenario

You continue building the **Grades Prototype**. Today you make slow operations (like report generation) run in the background with **async** so the UI doesn't freeze.

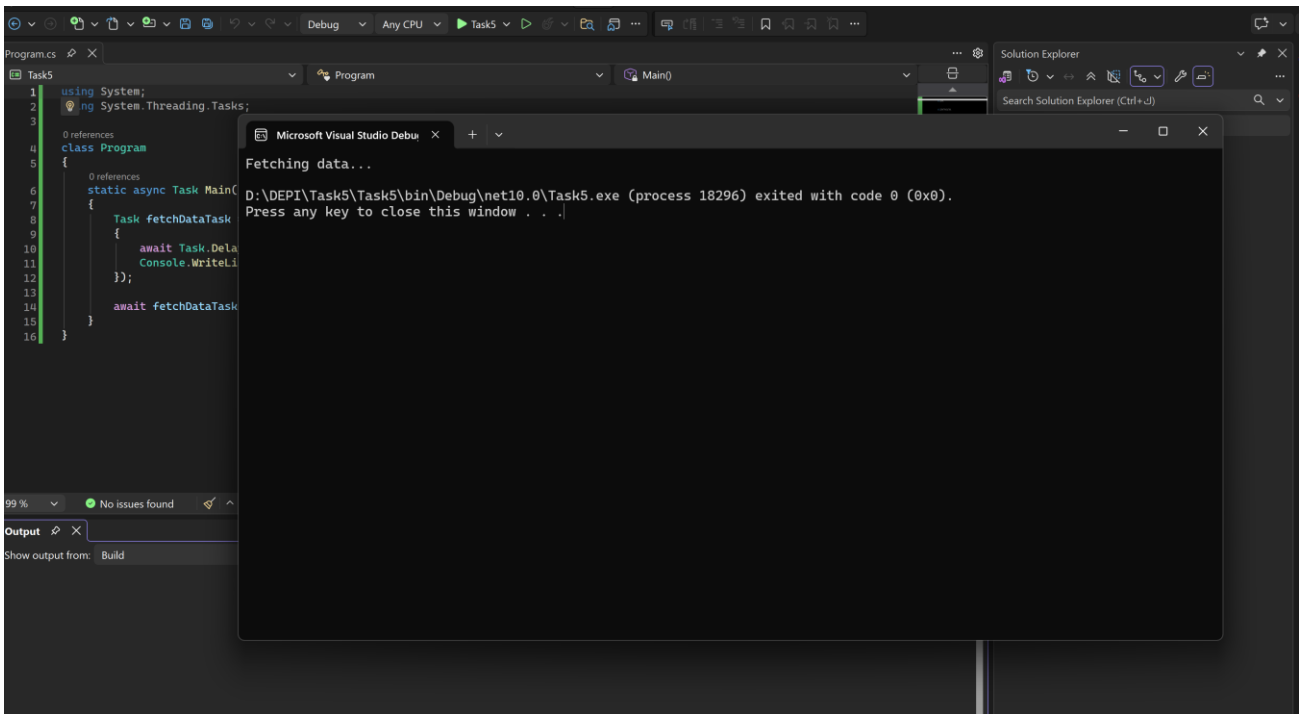
This assignment covers: **Tasks**, **async/await**, **lock**, and **IDisposable**.

Part 1 — Tasks (3 points)

1. Create a `Task` that prints "Fetching data..." after a `Task.Delay(1000)` delay, and run it.

```
using System;
using System.Threading.Tasks;
class Program
{
   static async Task Main()
   {
      Task fetchDataTask = Task.Run(async () =>
      {
         await Task.Delay(1000);
         Console.WriteLine("Fetching data...");
      });

      await fetchDataTask;
   }
}
```

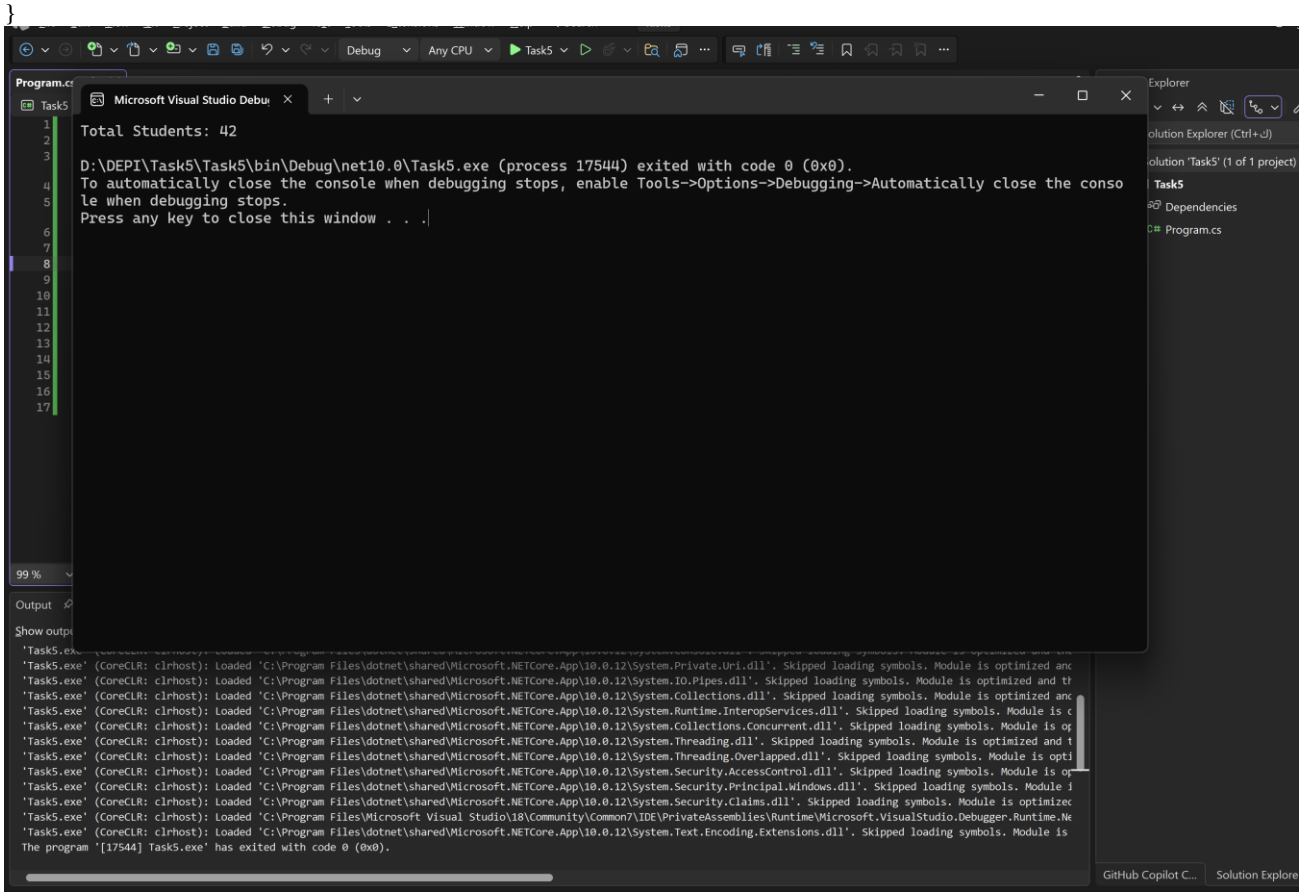


2. Create a `Task<int>` that returns a student count (a fixed number), and read the `.Result`.

```
using System;
```

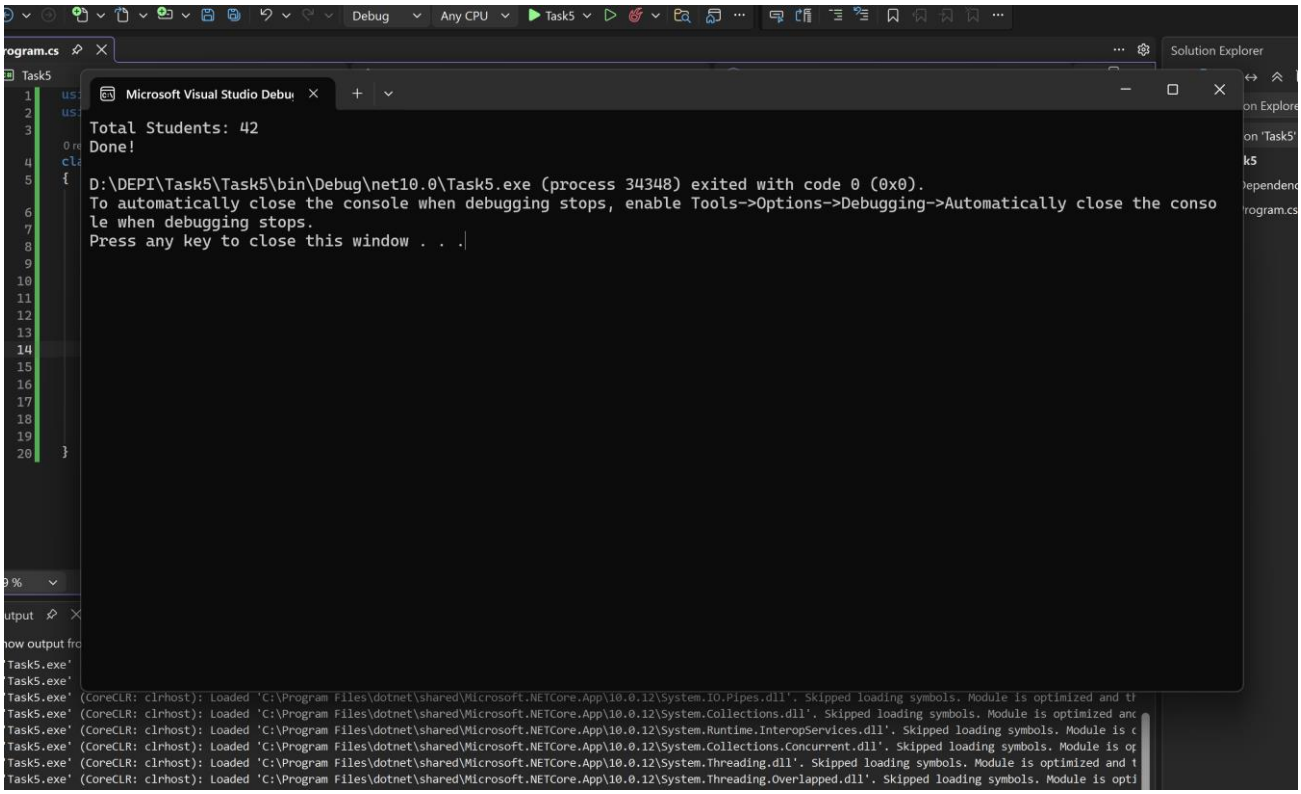
```
using System.Threading.Tasks;
class Program
{
   static void Main()
   {
      Task<int> getStudentCountTask = Task.Run(() =>
      {
         int studentCount = 42;
         return studentCount;
      });
      int count = getStudentCountTask.Result;

      Console.WriteLine($"Total Students: {count}");
   }
}
```



3. Use `ContinueWith` to print "Done!" after the task completes.

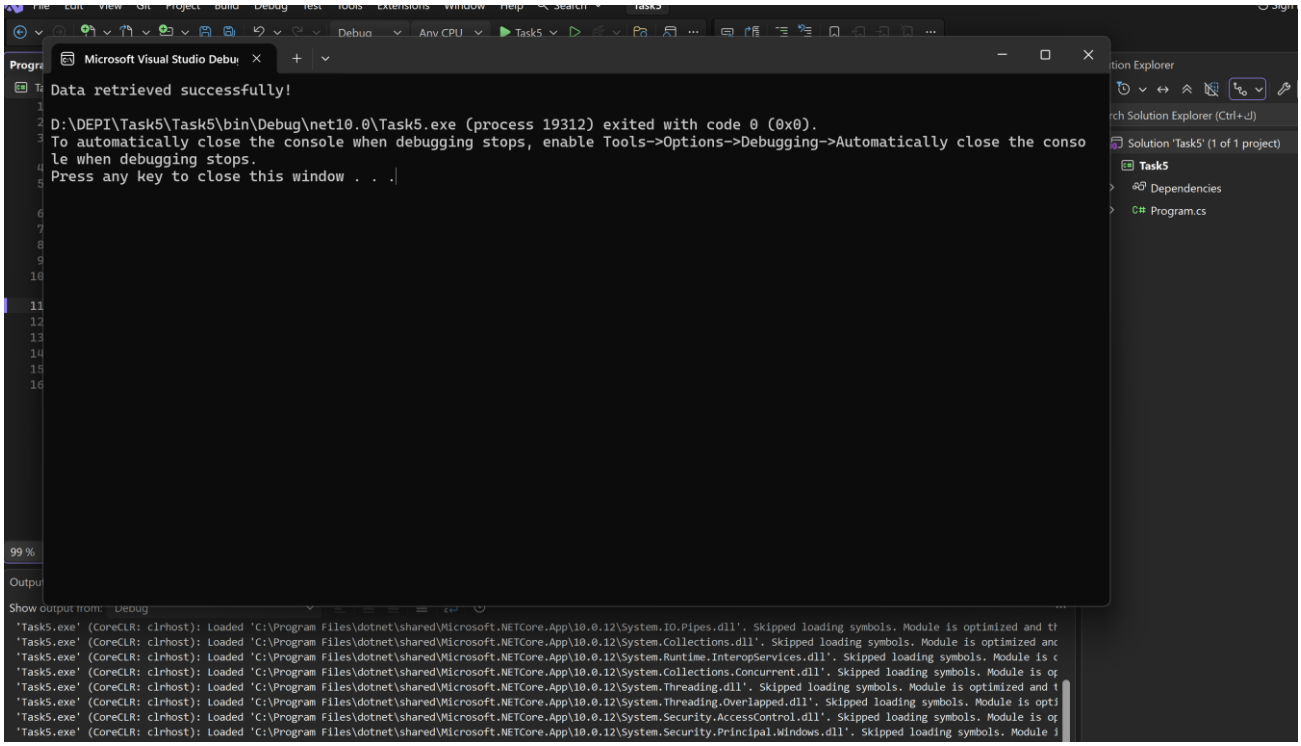
```
using System;
using System.Threading.Tasks;
class Program
{
   static void Main()
   {
      Task<int> getStudentCountTask = Task.Run(() =>
      {
         int studentCount = 42;
         return studentCount;
      });
      Task continuationTask = getStudentCountTask.ContinueWith(previousTask =>
      {
         Console.WriteLine("Done!");
      });
      Console.WriteLine($"Total Students: {getStudentCountTask.Result}");
      continuationTask.Wait();
   }
}
```



Part 2 — async/await (2 points)

4. Write a method `async Task<string> GetDataAsync()` with `await Task.Delay(500)` that returns a string.

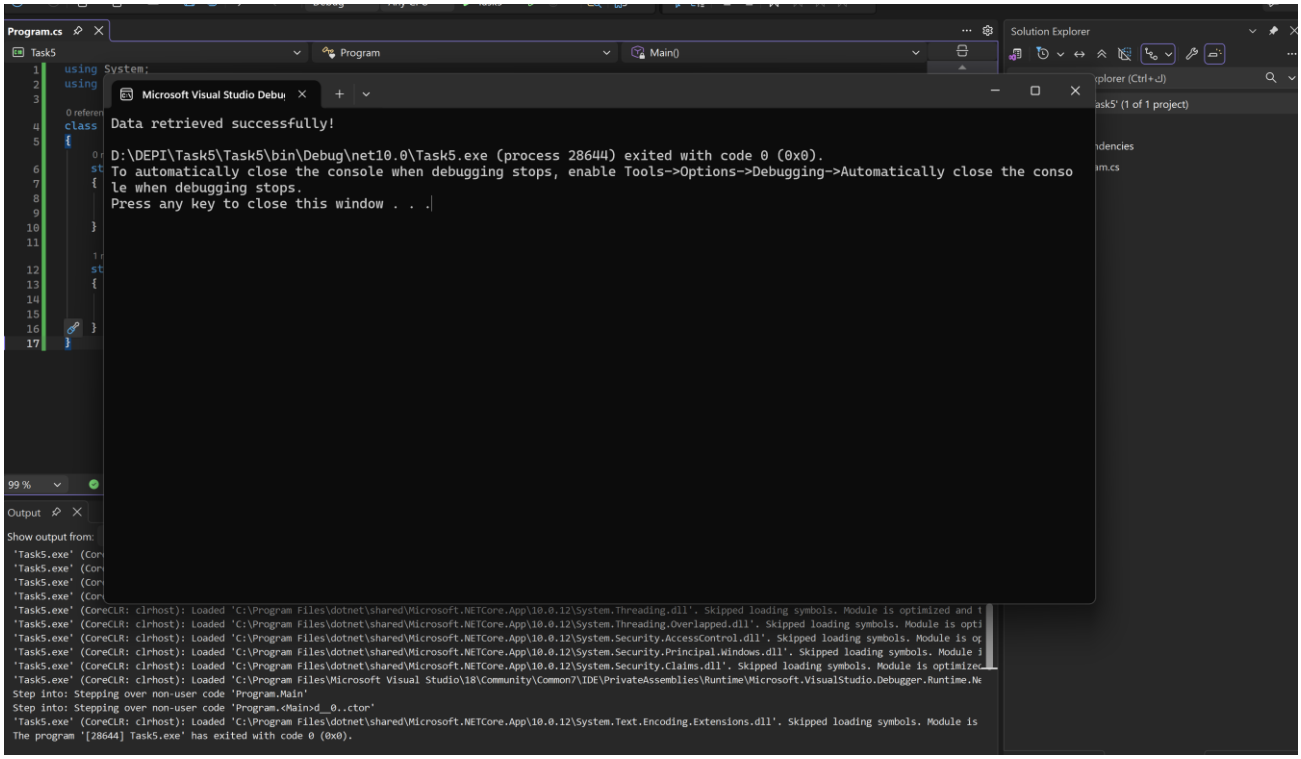
```
using System;
using System.Threading.Tasks;
class Program
{
   static async Task Main()
   {
      string data = await GetDataAsync();
      Console.WriteLine(data);
   }
   static async Task<string> GetDataAsync()
   {
      await Task.Delay(500);
      return "Data retrieved successfully!";
   }
}
```



5. Call it with `await` from `Main` (make `Main` async) and print the result.

```
using System;
using System.Threading.Tasks;
class Program
{
   static async Task Main()
   {
      string result = await GetDataAsync();
      Console.WriteLine(result);
   }

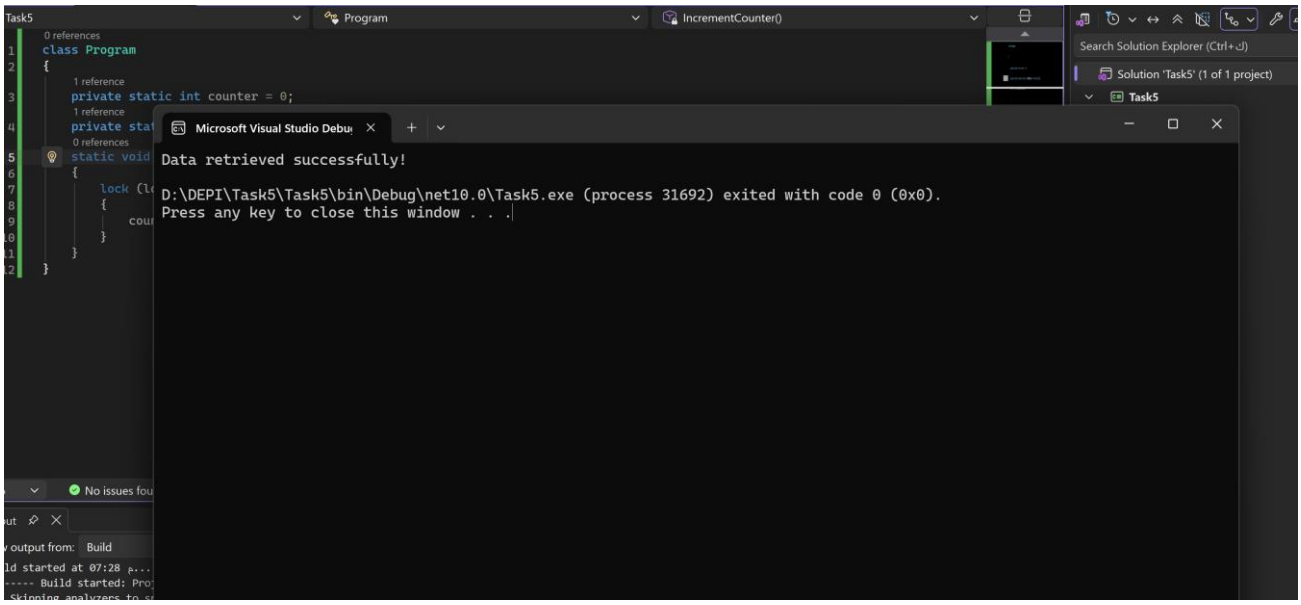
   static async Task<string> GetDataAsync()
   {
      await Task.Delay(500);
      return "Data retrieved successfully!";
   }
}
```



Part 3 — lock + Dynamic (3 points)

6. Create a shared counter `int counter` and a method that increments it inside a `lock` block.

```
class Program
{
   private static int counter = 0;
   private static readonly object lockObject = new object();
   static void IncrementCounter()
   {
      lock (lockObject)
      {
         counter++;
      }
   }
}
```



7. Question (in a comment): what is the difference between `var` and `dynamic`?

`var: Fixed type at compile-time. Cannot change. Example: var x = 10;`

`dynamic: Flexible type at runtime. Can change anytime. Example: dynamic y = 10; y = "Hello";`

 Submission: .cs file + output screenshots, posted to the group.

 Tip: **Working** code that you understand line by line is what matters — not perfect code. Use `breakpoints` if something doesn't run.